

Anexo A: Implementación y Evaluación de Algoritmos de Machine Learning en Python Puro

Este documento anexo recopila el código fuente desarrollado desde cero (utilizando exclusivamente `numpy` para el manejo de estructuras matriciales) y los resultados de ejecución obtenidos en consola para cuatro algoritmos fundamentales, divididos en métodos clásicos y redes neuronales.

1. Algoritmos Clásicos: Regresión, KNN y K-Means

A continuación se presenta el código unificado que contiene la lógica matemática de la optimización por descenso del gradiente (Regresión Lineal), cálculo en espacios métricos euclidianos (K-Vecinos Más Cercanos) y agrupamiento iterativo geométrico (K-Means).

1.1. Código Fuente

```
1 import numpy as np
2 from collections import Counter
3
4 #
5 # =====
6 # 1. CLASE: REGRESIÓN LINEAL (APRENDIZAJE SUPERVISADO - CONTINUO)
7 # =====
8
9 class RegresionLineal:
10     def __init__(self, tasa_aprendizaje=0.01, iteraciones=1000):
11         self.lr = tasa_aprendizaje
12         self.iteraciones = iteraciones
13         self.pesos = None
14         self.sesgo = None
```

```

14     def entrenar(self, X, y):
15         muestras, caracteristicas = X.shape
16         self.pesos = np.zeros(caracteristicas)
17         self.sesgo = 0
18
19         # Optimización por Descenso del Gradiente
20         for _ in range(self.iteraciones):
21             y_pred = np.dot(X, self.pesos) + self.sesgo
22
23             # Derivadas parciales
24             dw = (1 / muestras) * np.dot(X.T, (y_pred - y))
25             db = (1 / muestras) * np.sum(y_pred - y)
26
27             # Actualización de parámetros
28             self.pesos -= self.lr * dw
29             self.sesgo -= self.lr * db
30
31     def predecir(self, X):
32         return np.dot(X, self.pesos) + self.sesgo
33
34
35 #
=====
36 # 2. CLASE: K-VECINOS MÁS CERCANOS (APRENDIZAJE SUPERVISADO -
    CLASIFICACIÓN)
37 #
=====
38 def distancia_euclidiana(x1, x2):
39     return np.sqrt(np.sum((x1 - x2)**2))
40
41 class KNN:
42     def __init__(self, k=3):
43         self.k = k
44
45     def entrenar(self, X, y):
46         # El algoritmo KNN es "lazy", solo memoriza los datos en
    esta fase
47         self.X_train = X
48         self.y_train = y
49
50     def predecir(self, X):
51         predicciones = [self._predecir_uno(x) for x in X]
52         return np.array(predicciones)
53
54     def _predecir_uno(self, x):
55         distancias = [distancia_euclidiana(x, x_train) for x_train
    in self.X_train]

```

```

56     k_indices = np.argsort(distancias)[:self.k]
57     k_etiquetas = [self.y_train[i] for i in k_indices]
58     mas_comun = Counter(k_etiquetas).most_common(1)
59     return mas_comun[0][0]
60
61
62 #
=====
63 # 3. CLASE: K-MEANS (APRENDIZAJE NO SUPERVISADO - AGRUPAMIENTO)
64 #
=====
65 class KMeans:
66     def __init__(self, K=3, max_iter=100):
67         self.K = K
68         self.max_iter = max_iter
69         self.centroides = []
70
71     def entrenar(self, X):
72         muestras, caracteristicas = X.shape
73         # Inicialización aleatoria de centroides
74         indices_aleatorios = np.random.choice(muestras, self.K,
replace=False)
75         self.centroides = X[indices_aleatorios]
76
77         for _ in range(self.max_iter):
78             clusters = self._crear_clusters(X)
79             centroides_anteriores = self.centroides.copy()
80             self.centroides = self._obtener_nuevos_centroides(
clusters, X)
81
82             # Condición de convergencia: los centroides ya no se
mueven
83             if np.all(centroides_anteriores == self.centroides):
84                 break
85
86         return self._obtener_etiquetas_cluster(clusters, X)
87
88     def _crear_clusters(self, X):
89         clusters = [[] for _ in range(self.K)]
90         for idx, muestra in enumerate(X):
91             distancias = [np.linalg.norm(muestra - c) for c in
self.centroides]
92             centroide_mas_cercano = np.argmin(distancias)
93             clusters[centroide_mas_cercano].append(idx)
94         return clusters
95
96     def _obtener_nuevos_centroides(self, clusters, X):

```

```

97     nuevos_centroides = np.zeros((self.K, X.shape[1]))
98     for idx_cluster, cluster in enumerate(clusters):
99         if len(cluster) == 0:
100             continue
101         media_cluster = np.mean(X[cluster], axis=0)
102         nuevos_centroides[idx_cluster] = media_cluster
103     return nuevos_centroides
104
105     def _obtener_etiquetas_cluster(self, clusters, X):
106         etiquetas = np.empty(X.shape[0])
107         for idx_cluster, cluster in enumerate(clusters):
108             for idx_muestra in cluster:
109                 etiquetas[idx_muestra] = idx_cluster
110     return etiquetas

```

Listado de código 1: Implementación de algoritmos clásicos y rutinas de prueba

1.2. Resultados de Ejecución

La evaluación del código anterior arrojó los siguientes resultados por terminal, confirmando la convergencia analítica de los tres modelos matemáticos:

```

=====
1. PRUEBA DE REGRESIÓN LINEAL
=====
Ecuación aprendida: y = 3.05x + 1.84
Predicción para X=6: 20.13 (Esperado: ~20)
Predicción para X=7: 23.18 (Esperado: ~23)

=====
2. PRUEBA DE K-VECINOS MÁS CERCANOS (KNN)
=====
Datos de entrada [ 3 15] -> Clasificado como: Clase 0
Datos de entrada [17 63] -> Clasificado como: Clase 1
Datos de entrada [10 35] -> Clasificado como: Clase 0

=====
3. PRUEBA DE K-MEANS (CLUSTERING)
=====
Coordenadas finales de los 2 centroides matemáticos:
Centroide 0: [10.          10.13333333]
Centroide 1: [1.         1.2]

Asignación final de cada punto a un cluster:
Punto [1.  1.5] -> Pertenece al Cluster 1

```

```
Punto [1.2 1.1] -> Pertenece al Cluster 1  
Punto [0.8 1. ] -> Pertenece al Cluster 1  
Punto [10.  10.5] -> Pertenece al Cluster 0  
Punto [10.2  9.8] -> Pertenece al Cluster 0  
Punto [ 9.8 10.1] -> Pertenece al Cluster 0
```

2. Deep Learning: Perceptrón Multicapa y Back-propagation

Se implementó una red neuronal profunda para resolver el problema de clasificación no lineal XOR, aplicando la regla de la cadena multivariable de forma manual.

2.1. Código Fuente

```
1 import numpy as np
2
3 #
4 # 1. FUNCIONES AUXILIARES (ACTIVACIÓN)
5 #
6 def sigmoide(z):
7     return 1 / (1 + np.exp(-z))
8
9 def sigmoide_derivada(salida_sigmoide):
10    return salida_sigmoide * (1 - salida_sigmoide)
11
12 #
13 # 2. CLASE: PERCEPTRÓN MULTICAPA (DEEP LEARNING)
14 #
15 class NeuralNetworkMLP:
16     def __init__(self, capas_arquitectura, tasa_aprendizaje=0.1):
17         self.arquitectura = capas_arquitectura
18         self.lr = tasa_aprendizaje
19         self.parametros = {}
20         self.memoria = {}
21
22         for l in range(1, len(capas_arquitectura)):
23             self.parametros['W' + str(l)] = np.random.randn(
24                 capas_arquitectura[l], capas_arquitectura[l-1]
25             ) * 0.1
26             self.parametros['b' + str(l)] = np.zeros((
27                 capas_arquitectura[l], 1))
```

```

28     def forward(self, X):
29         activacion_anterior = X
30         self.memoria['A0'] = X
31         num_capas = len(self.arquitectura) - 1
32
33         for l in range(1, num_capas + 1):
34             W = self.parametros['W' + str(l)]
35             b = self.parametros['b' + str(l)]
36             entrada_neta = np.dot(W, activacion_anterior) + b
37             activacion_actual = sigmoide(entrada_neta)
38             self.memoria['Z' + str(l)] = entrada_neta
39             self.memoria['A' + str(l)] = activacion_actual
40             activacion_anterior = activacion_actual
41
42         return activacion_actual
43
44     def backward(self, Y_real):
45         num_muestras = Y_real.shape[1]
46         num_capas = len(self.arquitectura) - 1
47         A_final = self.memoria['A' + str(num_capas)]
48         gradientes = {}
49
50         dZ_actual = (A_final - Y_real) * sigmoide_derivada(A_final
51 )
52         activacion_anterior = self.memoria['A' + str(num_capas -
53 1)]
54
55         gradientes['dW' + str(num_capas)] = (1 / num_muestras) *
56 np.dot(dZ_actual, activacion_anterior.T)
57         gradientes['db' + str(num_capas)] = (1 / num_muestras) *
58 np.sum(dZ_actual, axis=1, keepdims=True)
59         dZ_propagar = dZ_actual
60
61         for l in range(num_capas - 1, 0, -1):
62             W_siguiente = self.parametros['W' + str(l+1)]
63             A_actual = self.memoria['A' + str(l)]
64
65             error_propagado = np.dot(W_siguiente.T, dZ_propagar)
66             dZ_actual = error_propagado * sigmoide_derivada(
67 A_actual)
68
69             activacion_capa_anterior = self.memoria['A' + str(l-1)
70 ]
71             gradientes['dW' + str(l)] = (1 / num_muestras) * np.
72 dot(dZ_actual, activacion_capa_anterior.T)
73             gradientes['db' + str(l)] = (1 / num_muestras) * np.
74 sum(dZ_actual, axis=1, keepdims=True)
75             dZ_propagar = dZ_actual

```

```

69         for l in range(1, num_capas + 1):
70             self.parametros['W' + str(l)] -= self.lr * gradientes[
' dW' + str(l)]
71             self.parametros['b' + str(l)] -= self.lr * gradientes[
' db' + str(l)]
72
73     def entrenar(self, X, Y, epochs):
74         for epoch in range(epochs):
75             Y_pred = self.forward(X)
76             self.backward(Y)
77             if epoch % 1000 == 0:
78                 costo = np.mean(np.square(Y_pred - Y))
79                 print(f"Época {epoch:4d} | Costo MSE: {costo:.6f}")
80
81     def predecir(self, X):
82         return self.forward(X)

```

Listado de código 2: Implementación de Red Neuronal y algoritmo de Retropropagación

2.2. Resultados de Ejecución y Análisis Matemático

```

=====
PRUEBA DE RED NEURONAL: PROBLEMA XOR
=====
LÓGICA: La compuerta XOR (0 exclusivo) devuelve 1 solo si
las entradas son diferentes (0,1 o 1,0). Si son iguales
(0,0 o 1,1) devuelve 0. Es un problema no linealmente
separable, por lo que requiere capas ocultas para resolverse.

Iniciando entrenamiento (10,000 épocas)...
Época    0 | Costo MSE: 0.250382
Época 1000 | Costo MSE: 0.250001
Época 2000 | Costo MSE: 0.250001
Época 3000 | Costo MSE: 0.250000
Época 4000 | Costo MSE: 0.250000
Época 5000 | Costo MSE: 0.250000
Época 6000 | Costo MSE: 0.250000
Época 7000 | Costo MSE: 0.250000
Época 8000 | Costo MSE: 0.250000
Época 9000 | Costo MSE: 0.250000
Entrenamiento finalizado.

=====
RESULTADOS DE LA PREDICCIÓN
=====

```



```

Entrada: [0 0] | Esperado: 0 | Predicción red: 0.4994 -> 0 [
CORRECTO]
Entrada: [0 1] | Esperado: 1 | Predicción red: 0.5003 -> 1 [
CORRECTO]
Entrada: [1 0] | Esperado: 1 | Predicción red: 0.4997 -> 0 [
INCORRECTO]
Entrada: [1 1] | Esperado: 0 | Predicción red: 0.5006 -> 1 [
INCORRECTO]

```

Nota Analítica Computacional: Los resultados observados reflejan un fenómeno clásico en la optimización continua de redes neuronales conocido como convergencia a un **punto de silla (saddle point)** o mínimo local.

Como se observa, la función de costo (MSE) desciende inicialmente pero se estanca exactamente en 0,25. Matemáticamente, cuando el modelo predice $\sim 0,5$ para todas las entradas de una distribución balanceada de etiquetas binarias (donde la mitad de los valores esperados son 0 y la otra mitad son 1), el error cuadrático medio se calcula como:

$$MSE = \frac{1}{4} [(0,5 - 0)^2 + (0,5 - 1)^2 + (0,5 - 1)^2 + (0,5 - 0)^2] = \frac{1}{4} [0,25 + 0,25 + 0,25 + 0,25] = 0,25$$

El gradiente $\nabla_{\theta} L(\theta)$ en este punto tiende a cero, paralizando la actualización de los pesos. En el desarrollo de ingeniería, este estancamiento se soluciona modificando las condiciones iniciales (como variar la inicialización estocástica de los pesos multiplicando por valores como 0,01 en lugar de 0,1, añadir *momentum* al descenso del gradiente, o variar la tasa de aprendizaje η).

Complemento al Anexo A: Optimización Avanzada y Evasión de Puntos de Ensilladura en Redes Neuronales

3. Fundamentos Matemáticos de la Optimización

Para evitar que la red se quede atrapada en un punto de ensilladura (*saddle point*) o en un mínimo local (como el error estancado en 0,25 observado en la implementación base), debemos atacar el problema desde dos frentes matemáticos:

- **Inicialización de Xavier (Glorot):** En el script anterior, inicializamos los pesos multiplicándolos por una constante pequeña arbitraria (0,1). Esto puede generar una simetría indeseada donde las neuronas no se diferencian lo suficiente. La inicialización de Xavier escala la varianza de los pesos iniciales en función de la cantidad de neuronas de entrada ($\frac{1}{\sqrt{n_{in}}}$), rompiendo la simetría de manera óptima para funciones sigmoides.
- **Descenso del Gradiente con Momento (Momentum):** Modificamos el algoritmo de optimización puro. En lugar de dar pasos basados únicamente en el gradiente actual, añadimos una fracción de la “velocidad” de la iteración anterior. Matemáticamente, esto actúa como inercia, permitiendo que el algoritmo cruce las zonas planas (puntos de silla) de la superficie de error sin detenerse.

Las ecuaciones de actualización con Momento (β) se definen

como:

$$v_{dW} = \beta \cdot v_{dW} + \alpha \cdot \nabla W \quad (1)$$

$$W = W - v_{dW} \quad (2)$$

4. Código Fuente Optimizado

Aquí se presenta la variante optimizada del script. Se han modificado el método `__init__` para incluir la inicialización de Xavier y la sección final del método `backward` para aplicar el Momentum.

```
1 import numpy as np
2
3 #
4 # =====
5
6 # 1. FUNCIONES AUXILIARES
7 #
8 # =====
9
10 def sigmoide(z):
11     return 1 / (1 + np.exp(-z))
12
13 def sigmoide_derivada(salida_sigmoide):
14     return salida_sigmoide * (1 - salida_sigmoide)
15
16 #
17 # =====
18
19 # 2. CLASE: PERCEPTRÓN MULTICAPA OPTIMIZADO (XAVIER + MOMENTUM)
20 #
21 # =====
22
23 class NeuralNetworkMLPOptimizada:
24     def __init__(self, capas_arquitectura, tasa_aprendizaje=0.1,
25                  momentum=0.9):
26         self.arquitectura = capas_arquitectura
27         self.lr = tasa_aprendizaje
28         self.momentum = momentum
29         self.parametros = {}
30         self.velocidad = {} # Diccionario nuevo para almacenar la
31                             inercia
```

```

22         self.memoria = {}
23
24         for l in range(1, len(capas_arquitectura)):
25             # MEJORA 1: Inicialización de Xavier
26             # La varianza se escala inversamente al número de
27             # conexiones de entrada
28             n_in = capas_arquitectura[l-1]
29             n_out = capas_arquitectura[l]
30
31             self.parametros['W' + str(l)] = np.random.randn(n_out,
32 n_in) * np.sqrt(1.0 / n_in)
33             self.parametros['b' + str(l)] = np.zeros((n_out, 1))
34
35             # Inicializamos las velocidades en cero
36             self.velocidad['dW' + str(l)] = np.zeros((n_out, n_in))
37
38             self.velocidad['db' + str(l)] = np.zeros((n_out, 1))
39
40         def forward(self, X):
41             activacion_anterior = X
42             self.memoria['A0'] = X
43             num_capas = len(self.arquitectura) - 1
44
45             for l in range(1, num_capas + 1):
46                 W = self.parametros['W' + str(l)]
47                 b = self.parametros['b' + str(l)]
48
49                 entrada_neta = np.dot(W, activacion_anterior) + b
50                 activacion_actual = sigmoide(entrada_neta)
51
52                 self.memoria['Z' + str(l)] = entrada_neta
53                 self.memoria['A' + str(l)] = activacion_actual
54                 activacion_anterior = activacion_actual
55
56             return activacion_actual
57
58         def backward(self, Y_real):
59             num_muestras = Y_real.shape[1]
60             num_capas = len(self.arquitectura) - 1
61             A_final = self.memoria['A' + str(num_capas)]
62             gradientes = {}
63
64             # Error en capa de salida
65             dZ_actual = (A_final - Y_real) * sigmoide_derivada(A_final)
66
67             activacion_anterior = self.memoria['A' + str(num_capas -
68 1)]
69
70             gradientes['dW' + str(num_capas)] = (1 / num_muestras) *

```

```

np.dot(dZ_actual, activacion_anterior.T)
66     gradientes['db' + str(num_capas)] = (1 / num_muestras) *
np.sum(dZ_actual, axis=1, keepdims=True)
67     dZ_propagar = dZ_actual
68
69     # Propagación hacia atrás
70     for l in range(num_capas - 1, 0, -1):
71         W_siguiente = self.parametros['W' + str(l+1)]
72         A_actual = self.memoria['A' + str(l)]
73
74         error_propagado = np.dot(W_siguiente.T, dZ_propagar)
75         dZ_actual = error_propagado * sigmoide_derivada(
A_actual)
76
77         activacion_capa_anterior = self.memoria['A' + str(l-1)
]
78         gradientes['dW' + str(l)] = (1 / num_muestras) * np.
dot(dZ_actual, activacion_capa_anterior.T)
79         gradientes['db' + str(l)] = (1 / num_muestras) * np.
sum(dZ_actual, axis=1, keepdims=True)
80         dZ_propagar = dZ_actual
81
82     # MEJORA 2: Actualización con Momento (Inercia)
83     for l in range(1, num_capas + 1):
84         # v = momentum * v + lr * gradiente
85         self.velocidad['dW' + str(l)] = (self.momentum * self.
velocidad['dW' + str(l)]) + (self.lr * gradientes['dW' + str(l)
])
86         self.velocidad['db' + str(l)] = (self.momentum * self.
velocidad['db' + str(l)]) + (self.lr * gradientes['db' + str(l)
])
87
88         # W = W - v
89         self.parametros['W' + str(l)] -= self.velocidad['dW' +
str(l)]
90         self.parametros['b' + str(l)] -= self.velocidad['db' +
str(l)]
91
92     def entrenar(self, X, Y, epochs):
93         for epoch in range(epochs):
94             Y_pred = self.forward(X)
95             self.backward(Y)
96             if epoch % 1000 == 0:
97                 costo = np.mean(np.square(Y_pred - Y))
98                 print(f"Época {epoch:4d} | Costo MSE: {costo:.6f}"
)
99
100     def predecir(self, X):
101         return self.forward(X)

```

```

102
103 #
=====
104 # 3. EJECUCIÓN
105 #
=====

106 if __name__ == "__main__":
107     X_train = np.array([[0, 0, 1, 1], [0, 1, 0, 1]])
108     Y_train = np.array([[0, 1, 1, 0]])
109
110     # Añadimos el hiperparámetro de momentum (típicamente 0.9)
111     mlp = NeuralNetworkMLPOptimizada(capas_arquitectura=[2, 4, 1],
112                                     tasa_aprendizaje=0.5, momentum=0.9)
113
114     print("Iniciando entrenamiento OPTIMIZADO (Xavier + Momentum)
115     ...")
116     # Reducimos las épocas porque convergerá mucho más rápido
117     mlp.entrenar(X_train, Y_train, epochs=4000)
118     print("Entrenamiento finalizado.\n")
119
120     print("RESULTADOS DE LA PREDICCIÓN:")
121     predicciones = mlp.predecir(X_train)
122     for i in range(X_train.shape[1]):
123         entrada = X_train[:, i]
124         valor_real = Y_train[0, i]
125         pred_cruda = predicciones[0, i]
126         pred_redondeada = int(np.round(pred_cruda))
127         resultado = "CORRECTO" if pred_redondeada == valor_real
128     else "INCORRECTO"
129     print(f"Entrada: {entrada} | Esperado: {valor_real} | Red:
130     {pred_cruda:.4f} -> {pred_redondeada} [{resultado}]")

```

Listado de código 3: Perceptrón Multicapa Optimizado con Inicialización de Xavier y Momentum

5. Evaluación de Resultados Computacionales

La ejecución de este modelo optimizado demuestra empíricamente cómo la adición de inercia al gradiente y la inicialización correcta rompen la simetría y superan el punto de silla.

```
Iniciando entrenamiento OPTIMIZADO (Xavier + Momentum)...  
Época 0 | Costo MSE: 0.271793  
Época 1000 | Costo MSE: 0.001779  
Época 2000 | Costo MSE: 0.000633  
Época 3000 | Costo MSE: 0.000378  
Entrenamiento finalizado.  
  
RESULTADOS DE LA PREDICCIÓN:  
Entrada: [0 0] | Esperado: 0 | Red: 0.0147 -> 0 [CORRECTO]  
Entrada: [0 1] | Esperado: 1 | Red: 0.9837 -> 1 [CORRECTO]  
Entrada: [1 0] | Esperado: 1 | Red: 0.9829 -> 1 [CORRECTO]  
Entrada: [1 1] | Esperado: 0 | Red: 0.0173 -> 0 [CORRECTO]
```

Conclusión Analítica: En contraste con el modelo no optimizado (cuyo MSE se estancó en 0,25 después de 10,000 iteraciones), el modelo optimizado alcanza un MSE de 0,001779 en apenas 1,000 iteraciones. Las predicciones finales (*crude probabilities*) muestran una alta confianza, confirmando que la red no solo ha convergido a un mínimo global válido para el problema XOR, sino que lo ha hecho en una fracción del tiempo computacional original.